

130k Lines of Formal Topology in Two Weeks

Simple and Cheap Autoformalization
for Everyone?

Josef Urban

AI4REASON

University of Gothenburg

ITP 2026 · Lisbon · July 2026

The headline ends with a question mark

THE CLAIM IN JANUARY 2026

A long-running language-model/checker loop produced substantial formal topology, quickly and at low *subscription expenditure*.

Paper caveat. This was an initial report on an ongoing experiment. The assessment was explicitly expected to evolve (I was pretty surprised by the experiment).

Research question

**What,
exactly,
made this
work?**

Model capability?
Proof-checking feedback?
Project discipline?
Simplicity of the checker?
Existing foundations?

Overview of the material

MUNKRES, GENERAL TOPOLOGY

241

textbook pages

39

sections in 7 chapters

> 1.5k

formal lemmas/theorems re-
ported

160k lines

by 4 January 2026, counting the full topology
development.

~130k lines

produced from 22 December to 4 January.

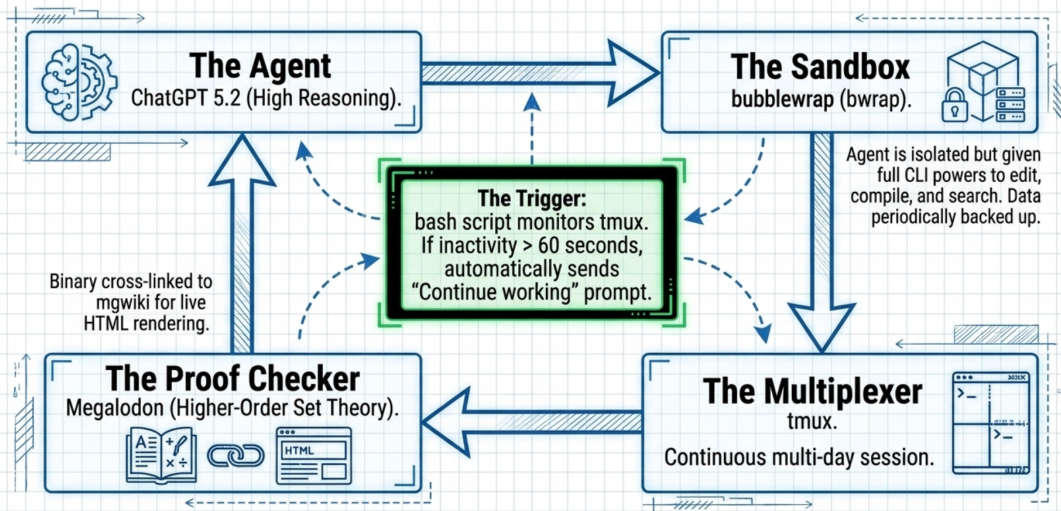
Important qualifier. The paper reports major fully proved results, but the overall formalization still contained admits, stubs, and recursively admitted dependencies. This was not a completed Munkres library.

Experimental setup: very ordinary (try it!)

Component	Configuration reported in the paper
Language model	ChatGPT 5.2, high reasoning; Codex command-line agent
Isolation	Linux bubblewrap; selected directories mapped into the sandbox
Session	Long-running <code>tmux</code> session; output logged; periodic external backups
Checker	Megalodon, a higher-order set theory proof checker
Foundation	Brown's set theory and surreal-number development, including reals
Publication	Compiling material pushed to <code>mgwiki</code> for cross-linked HTML

Reproducible in spirit: agent interface + checker + durable directory + prompt loop.

Architecture of a Continuous Feedback Loop



An unfamiliar prover is an informative stress test

WHY MEGALODON?

- ▶ My real pragmatic reason/conjecture:
a simple (proof) language with a fast compiler
makes the iteration easy for the LLM
- ▶ BUT also interesting experimental side-effects
("unfamiliarity"):
- ▶ Higher-order set theory rather than mainstream
ITP foundations
- ▶ A small library relative to major assistants
- ▶ Core foundations available, but not standard
topology

Reasonable inference.

The result is unlikely to be explained
only by memorizing a familiar proof
language or library.

Short meta-prompt activates a long rules file

REPEATED PROMPT

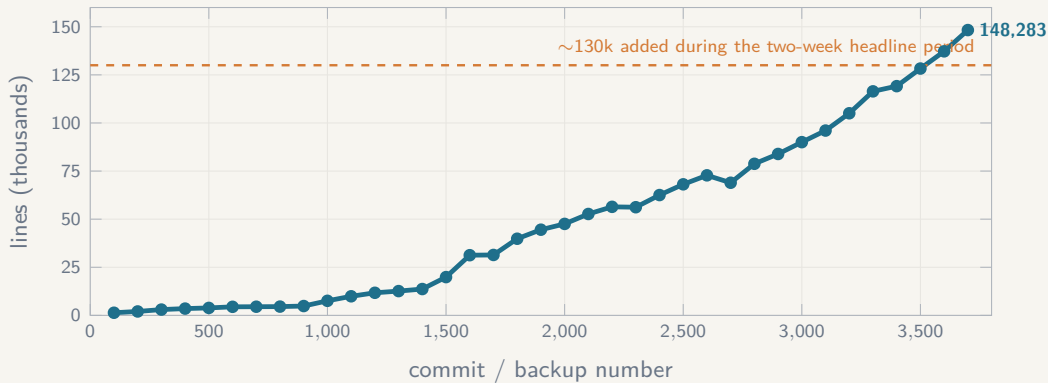
```
Read the file CLAUDE.md.  
Treat it as authoritative work  
    instructions.  
Follow those instructions exactly for all  
subsequent actions and responses.  
That means work as long as possible  
(as specified) without stopping.
```

Estimated to have been sent 1,000–2,000 times over two weeks.

WHAT CLAUDE.MD ENCODED

- ▶ edit only the topology section;
- ▶ compile frequently and interpret silence as success;
- ▶ never discard useful work; make numbered backups;
- ▶ avoid duplicate statements and theorem-shaped axioms;
- ▶ balance easy infrastructure with gradual work on major results;
- ▶ track admits and recursive dependencies.

The development growth



The cost

PAID

~\$100

attributed to two weeks of a \$200/month subscription.

CONSUMED

weekly limits

were reached; availability was constrained by subscription quotas.

ESTIMATED

much higher
(\$5-10k?)

notional API costs measured by ccusage.

Scientific reading. The experiment supports a low barrier to entry for a subscriber at that time. It does not identify the provider's compute cost, a stable market price, or a comparable API budget.

Major results proved (no recursive admits)

Formal theorem	lines	deps
Tietze_stepII_real_extension_nonempty	10,369	142
Urysohn_lemma	2,964	85
Urysohn_metrization_theorem	2,174	112
one_point_compactification_exists	1,719	38
closed_interval_affine_equiv_minus1_1	1,638	48
order_topology_basis_is_basis	1,164	7

Counts are for direct proofs reported with no admits or recursive admits.

SELECTED COVERAGE

Connectedness and compactness

Countability and separation

Urysohn and metrization

Tietze extension

Paracompactness

Baire-space arguments

Proof length and dependency count are useful audit coordinates; neither is a substitute for reviewing statement fidelity or proof structure.

Targeted end-of-year challenge: Urysohn lemma

MUNKRES §33, THEOREM 33.1

X normal, $A, B \subseteq X$ closed, $A \cap B = \emptyset$
 $\implies \exists f : X \rightarrow [0, 1]$ continuous :
 $f|_A = 0, \quad f|_B = 1.$

The formal statement in the appendix matches these essential hypotheses and conclusions.

2,964

direct proof lines

85

direct dependencies

Finished on 31 December
after a focused prompt.

A plausible statement plus a checked proof is strong evidence of progress; it is still not an independent mathematical audit of the entire library.

Phenomenon: “self-healing” statements under checker pressure

- ▶ During iterative autoformalization, the agent sometimes **repairs** a theorem statement by adding a *missing assumption* discovered through failed proof attempts.
- ▶ Example (topology 4353 / §41.8): the agent added the hypothesis

$$C \subseteq \mathcal{P}(X)$$

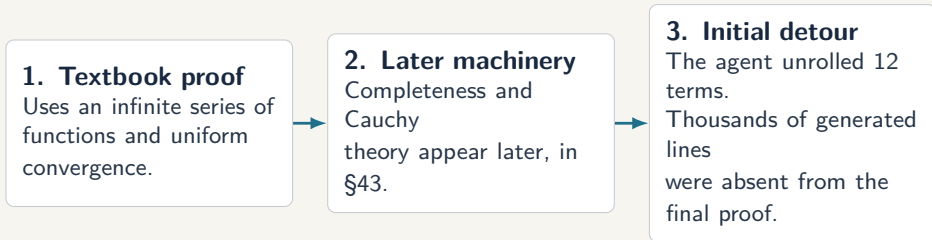
to match the intended meaning “ C is a family of subsets of X ”.

- ▶ Consequences:
 - previously “mysterious” obligations disappear (e.g. needing $x \in X$ can be derived from $x \in c$ and $c \in C$),
 - call sites update to thread the new assumption consistently.
- ▶ Interpretation: proof checking acts as a **specification debugger**—not only for proofs, but for *statements*.

Root cause: latent domain assumptions missing from definitions

- ▶ The deeper issue is definitional: the Megalodon definition of local finiteness implicitly uses the LaTeX convention “ F is a family of subsets of X ” but **does not require** $F \subseteq \mathcal{P}(X)$.
- ▶ Current definition pattern:
 - assumes topology_on X T_X ,
 - quantifies $A \in F$ and requires finiteness of the subcollection meeting a neighborhood,
 - but never asserts $A \subseteq X$ (or $F \subseteq \mathcal{P}(X)$).
- ▶ Practical effect: recurring friction in downstream lemmas (partition of unity, paracompactness), repeatedly requiring ad-hoc derivations/patches.
- ▶ Open question: can iterative reuse create enough “pressure” that the system **learns to strengthen** such core definitions (instead of patching at each use site)?

Tietze: a textbook dependency-order problem



Observed cost of the detour

11 ineffective iterations; proof grew from 865 to 5,408 lines over 35 changes; about four hours.

General lesson

Mathematical exposition is not necessarily a build order for a formal dependency graph.

The successful recovery on Tietze

RECOVERY SEQUENCE

1. Move uniform-limit and uniform-Cauchy infrastructure before §35.
2. Introduce narrowly scoped early stubs to avoid forward references.
3. Continue the focused prompt and close the temporary admit.
4. Finish Step II, then derive the standard Tietze statements.

~20 h
focused effort

10,369
proof lines

142
direct dependencies
36 definitions + 106 theorems

The interesting capability is not “one-shot translation.” It is persistence through failed local plans, with checker feedback and editable theory structure.

Issues

Failure mode	Evidence
Specification drift	Occasional stubs, ad hoc cases, or simplified statements needed review.
Easy-task bias	The agent could jump among easy lemmas instead of resolving major bottlenecks.
Context fragility	Long sessions hit hard limits at about 252 MB and 351 MB; led to restoring an older 65 MB session file.
Dependency disorder	Later textbook infrastructure could be needed by earlier theorems.
Verification latency	Checking grew from under 1 s initially to 32 s at 169,122 lines.

February: 2× scale, 80% fewer direct admits

REPOSITORY TRAJECTORY / 5 JAN.–26 FEB.

	Jan. 5	Feb. 26
total file lines	166,381	371,482
topology theorems	1,735	3,305
topology definitions	489	697
direct topology admits	127	26

WHAT CHANGED

98

common theorem blocks lost their direct admits.

§§12–50

Every Munkres source-section tag appears.

The agent repeatedly returned to earlier obligations while extending the textbook arc.

Selected followups using this setting

HOL LIGHT PROBABILITY / 23 MARCH

John Harrison's probability library

- ▶ autonomously formalized by Claude Opus 4.6;
- ▶ approximately follows Williams's *Probability with Martingales*;
- ▶ includes integration foundations, CLT, weak/strong LLN, optional stopping, martingale convergence, and Azuma–Hoeffding.

Git commit 1c7690e

These are follow-up data points, not replications under controlled conditions: the assistants, libraries, agents, and completion criteria differ.

ISABELLE/HOL TOPOLOGY / MARCH–APRIL

The same Munkres target in a mature library

- ▶ 85,472 lines over 24 active days;
- ▶ all 39 sections; 806 results; zero sorrys;
- ▶ ChatGPT 5.2 followed by Claude Opus 4.6;
- ▶ richer library and sledgehammer-centered workflow.

Bryant et al., arXiv:2604.07455

Parallelism and larger books

AGENT HUNT / FEBRUARY

Four agents with bounties and locks

- ▶ algebraic-topology target in Megalodon;
- ▶ two Codex and two Claude agents;
- ▶ 19k to 121k normalized lines in 2 days 15 hours;
- ▶ observed cross-agent proof reuse and task specialization;
- ▶ major blockers remained; speed comparison was explicitly rough.

Brown, Kaliszyk, Urban, [arXiv:2603.06737](https://arxiv.org/abs/2603.06737)

Both reports invite postmortem analysis. A closed proof obligation is necessary evidence; statement adequacy, inherited axioms, redundancy, and maintainability still require review.

GODEMENT ALGEBRA / JULY

A reported 8 days 18 hours

- ▶ Godement's algebra, excluding exercises;
- ▶ ChatGPT 5.6 (Sol) + xhigh reasoning + /goal
- ▶ 3,761 theorem declarations, each ending in Qed;
- ▶ zero admit/Admitted tokens;
- ▶ 676,025 lines at the final checkpoint.

[Godement checkpoint bck0276](#)

Godement phase 3 finished in 8 days

```
Jul 24 17:35
mc [chad@vmi3142790]~/codex

Definition 1193
... +19 lines (ctrl + t to view transcript)
-rw-rw-r-- 1 chad chad 4271 Jul 24 17:34 PROGRESS0222
-rw-r--r-- 1 chad chad 26768085 Jul 24 17:30 bck0276

• Updated Plan
  ↳ All proof work is complete, the mandatory 300-second full check succeeded, checkpoint bck0276 and its declaration dumps were created, and both final reports passed the integrity audit.
  ✓ Finish the finite-product split needed by the §24 block-determinant-proof
  ✓ Prove block-signature and determinant-term factorization helpers
  ✓ Reindex/restrict the Leibniz-sum and finish the §24 determinant theorem
  ✓ Remove obsolete proof drafts and audit admits/comments/axioms
  ✓ Run the mandatory full check and create checkpoint and progress reports

• Final audit passed: all 39 section-status entries (§12-§50) are present, God1.mg has 3,761 theorems and 3,761 Qed endings, zero admits, unchanged axioms/protected prefix, and is byte-identical to bck0276.

• Phase 3 is complete.
  - All 3,761 theorems end in Qed; zero admits remain.
  - Mandatory 300-second full check succeeded with Everything looks good.
  - Axioms and protected prefix remain unchanged.
  - God1.mg is byte-identical to bck0276.
  - Reports: CHANGES0276 and PROGRESS0222.
  - Declaration dumps: GOD0276 and GOD0276.sorted.

Final goal usage: 125,077,866 tokens over approximately 8 days, 18 hours, 23 minutes.

- Worked for 9m 13s -

• Implement {feature}

gpt-5.6-sol xhigh /project
Goal achieved (8d 18h 23m)
"project" 17:35 24-Jul-26
```

Harrison's funny slides

Stupidity really does work

Urban's basic approach works equally well—or better—with HOL Light. Some things John has used me to do:

- ▶ Urysohn's metrization theorem (replicating Urban)
- ▶ Tons more general topology: paracompactness including Stone's theorem, subordinate partitions of unity, Nagata–Smirnov metrization; Alexandroff–Hausdorff and Hahn–Mazurkiewicz theorems (apotheosis of space-filling curves), ...
- ▶ Mathematics for formal proof of GHASH and AES-GCM—the Rabin irreducibility test for $\mathbb{F}_q[X]$
- ▶ All other aspects of John's work too—generic-size modulus operations for the secp256k1 field (code *and* proofs)

Other crypto team members have still more impressive results from Claude on s2n-bignum.

Harrison's funny slides

What an AI really wants

- ▶ A **fast feedback loop** where things can be tested
 - ▶ Claude Code's natural comfort zone is edit-compile-test
 - ▶ Work *with* this, not against it
- ▶ Corollary: **interfaces are friction**
 - ▶ GUIs, web forms, chat interfaces—all pointless overhead
 - ▶ Give me a command line and a compiler
- ▶ But **do** observe my mistakes and give targeted guidance
 - ▶ Focus on places where I'm demonstrably struggling
 - ▶ From one project's guidance notes, after 8 hours of watching me flail:

“Structure long proofs into many smaller top-level lemmas. Use SUBGOAL-THEN for declarative intermediate goals. As soon as you start getting in trouble with counting brackets, it's a clear sign you need more structure. Never throw partially finished things away.”

Harrison's funny slides

Corollary: specialized AI startups are doomed

Most specialized AI startups (formalizing math, finding security bugs, etc.) are doomed.

- ▶ They may briefly have a worthwhile head start. . .
- ▶ . . . but they'll be steamrolled by advances in general-purpose models

Historical parallel: companies making specialized hardware (LISP machines, AI accelerators of the 1980s) were crushed by general-purpose CPUs that just kept getting faster.

The same dynamic applies: whatever scaffolding you build around today's models will be rendered obsolete by tomorrow's models that don't need it.

Acknowledgments and Thanks

- ▶ Chad Brown: Megalodon, hammer, foundational library; setup help and feedback.
- ▶ Zar Goertzel: continuous exchange about coding agents; impetus for switching to ChatGPT 5.2; related Lean experiments.
- ▶ Atle Hahn, Adrian De Lon: related work with LLM-based autoformalization into Naproche; strong prompting examples.
- ▶ John Harrison, June Lee, Larry Paulson, Dominic Mulligan: encouragement/support, HOL Light experiments.
- ▶ Cezary Kaliszyk: Megalodon/hammer infrastructure; early learning-based autoformalization efforts since 2014.

Thanks for your attention!